



PROJET DE DIPLÔME, EN PHYSIQUES DE L'INGENIEUR , PREMIER NIVEAU
STOCKHOLM , SUÈDE 2015

L'apprentissage par renforcement et le Jeu de Nim

PAUL GRAHAM ET WILLIAM SEIGNEUR

KTH INSTITUT ROYAL DE TECHNOLOGIE

CSC, INFORMATIQUE

Apprentissage par renforcement et le jeu de Nim

Paul Graham

prgraham@kth.se

Guillaume Seigneur

wlord@kth.se

Projet de Diplôme en Ingénierie Physique

Premier Cycle (SA104X, 15 ECTS)

05-2015

École d'informatique (CSC)

Institut Royal de Technologie

Stockholm

Superviseur : Pawel Herman (CSC)

Examineur : Mårten Olsson (SCI)

Abstrait

Cet article traite du concept d'apprentissage par renforcement (RL) appliqué à trouver la stratégie gagnante du jeu mathématique Nim. Deux

Les algorithmes Q-learning et SARSA ont été comparés à l'aide de plusieurs différents ensembles de paramètres dans trois régimes de formation différents. Une analyse sur l'évolutivité a également été entreprise.

Il a été constaté que le réglage des paramètres pour l'optimalité est difficile et beaucoup de temps, mais les agents RL ont appris une stratégie gagnante, en essentiellement le même temps pour les deux algorithmes. Quant à l'évolutivité, elle a montré que l'augmentation du temps d'apprentissage est effectivement un problème dans ce domaine. approche.

La pertinence des différents régimes de formation ainsi que d'autres les aspects conceptuels de l'approche sont discutés.

Nous concluons que cette utilisation de RL est une méthode prometteuse, même si laborieux à optimiser dans ce cas et devient vite inefficace lors de l'agrandissement du problème. Des idées sont discutées et proposées pour recherches futures sur la résolution de ces facteurs limitants.

Résumé

Ce rapport utilise le concept d'apprentissage par renforcement (RL) et

Les agents RL se préparent à vous aider à déterminer la stratégie marketing et à le faire

matematiska spelet Nim. Cet algorithme, Q-learning et SARSA, avec

flera olika paramètreinställningar jämfördes and tre olika träningsregimer.

Därutöver analyseades efficaces av storleksökning av spelet.

Je comprends la visa det sig att bestämmandet av parametrar för

et c'est optimal entre l'heure actuelle et l'heure actuelle, même pour le RL-

agent avec des paramètres de funna lyckades lära sig den vinnande

stratégie, et de nombreux algorithmes sont utilisés pour réussir.

Att ökningen av inlärningstid vid växande uppgifter är ce problème

verifierades också i undersökningen.

Relevansen av de olika träningsregimerna behandlas, likväl andra

concept frågor.

Certaines salopes peuvent s'amuser en s'amusant avec RL et en les aimant

méthodes pour que les hommes optimisent et med nackdelen att den lätt

blir ineffektiv vid större problème. Je rapporte des diskuterar idéer om

föreslagen forskning på lösningar jusqu'au début du facteur.

Contenu

1. Introduction	1
1.1 Énoncé du problème	2
1.2 Portée	3
2 Théorie et contexte	3
2.1 Apprentissage par renforcement	3
2.2 Processus de décision de Markov	4
2.3 Algorithmes d'apprentissage par renforcement	5
2.3.1 Q-Learning	5
2.3.2 SARSA	7
2.4 Nim	8
2.4.1 Somme numérique binaire et stratégie gagnante de Nim	9
3 Méthodologie	dix
3.1 Environnement, états et actions	dix
3.2 L'agent	dix
3.3 Formation de l'agent	dix
3.3.1 Agent contre Smart	11
3.3.2 Agent contre aléatoire	12
3.3.3 Agent contre agent	12
3.4 Évaluation de l'agent	12
3.5 Mise à l'échelle	12
4 Résultats et discussion	13
4.1 Sélection des paramètres	13
4.2 Agent contre Smart	16
4.3 Agent contre aléatoire	16
4.4 Agent contre agent	17
4.5 Mise à l'échelle	18
4.6 Autres observations et remarques	22
5. Conclusion	23
6 références	25

1. Introduction

Depuis la création du premier ordinateur, son importance pour l'humanité s'est rapidement accrue. Par facilitant les calculs et les simulations, les ordinateurs propulsent le développement technologique. Comme la plupart des choses que nous utilisons dans notre vie quotidienne ont été numérisées, notre société moderne a deviennent dépendants des systèmes informatiques.

Aussi fascinée que soit l'humanité par le pouvoir de son propre esprit, un intérêt croissant s'est manifesté s'est présenté dans ce qu'est l'intelligence et si nous pouvons la créer. En essayant de répondre à ces questions Les domaines scientifiques connaissent une croissance rapide et couvrent la modélisation des réseaux neuronaux et du cerveau. calculs inspirés [1, 2, 3].

En psychologie, la théorie du behaviorisme décrit comment le comportement animal peut être modifié à la suite d'un renforcement dû à une récompense ou à une punition. De manière analogue à ceci [1], une branche de l'intelligence artificielle s'est développée qui utilise les récompenses pour permettre un agent apprend à effectuer une certaine tâche. Cette méthode est appelée Apprentissage par Renforcement (RL) et sera étudié dans cet article.

RL est une méthode de programmation d'un ordinateur (l'agent) pour qu'il puisse apprendre comment effectuer des tâches de manière autonome. L'apprentissage peut s'effectuer soit sous la supervision d'un humain (apprentissage supervisé) ou l'agent peut être laissé seul pour explorer son environnement (apprentissage non supervisé) [4]. Pour permettre à l'agent d'apprendre, des récompenses sont données pour encourager le agent pour trouver la solution optimale à une tâche donnée.

RL offre un vaste potentiel au domaine de l'intelligence artificielle, avec des opportunités pour comportement humain dans les robots, algorithmes de recherche et plus encore. Un exemple de ressemblance humaine Le comportement des robots est celui du bras robotique à couple contrôlé qui apprend la tâche de retourner une crêpe. [5]. Cette tâche, qui peut paraître triviale, est difficile à réaliser même pour certains humains et même plus difficile à implémenter dans le code, a été implémenté avec succès à l'aide de RL. Cependant, le but de RL ne consiste pas nécessairement à imiter le comportement humain. Le fait que l'agent s'apprenne par lui-même les essais et erreurs signifient qu'il essaiera de nombreuses stratégies différentes avant de trouver la meilleure. Ce peut ouvrir la voie à des solutions jusqu'alors inconnues, comme dans le cas du premier agent RL TD-gammon qui a proposé ses propres stratégies complètement nouvelles pour jouer au backgammon [6].

Les applications de l'intelligence artificielle sont déjà largement répandues. Ce qui est unique dans RL cependant, c'est qu'il ne nécessite pas de programmation explicite de chaque situation [7], mais plutôt de l'agent apprend de zéro à accomplir une tâche. Par conséquent, des tâches complexes peuvent être apprises avec encore moins de programmation. De plus, cela crée un agent beaucoup plus flexible qu'un agent codé.

avec un seul objectif puisque les agents peuvent ainsi apprendre à effectuer davantage de tâches dans de nombreux domaines différents. façons [8] ainsi que comment effectuer des tâches que les humains ne savent peut-être pas décrire dans le code [5].

L'un des axes de la recherche contemporaine en RL (et de cet article) est de permettre aux agents d'apprendre à jouer à des jeux. Un exemple récent publié dans la revue Nature est le travail réalisé par Deep Pensez à un agent apprenant à jouer à 49 jeux Atari classiques [8].

Cet article se concentrera sur le jeu mathématique Nim. C'est intéressant car c'est plutôt jeu simple, mais représentatif des jeux combinatoires [9] et a une solution connue [10]. Avoir une solution connue facilite la mesure des performances.

L'applicabilité de RL à Nim a déjà fait l'objet de recherches [11], cependant le la faisabilité de l'utilisation de RL avec Nim à mesure que les dimensions du jeu augmentent n'a pas été étudiée. L'évolutivité est un problème connu de RL [7]. Comme le nombre de degrés de liberté de l'agent augmente, le temps nécessaire à l'agent pour terminer sa période de formation augmente également, ce qui est connue sous le nom de malédiction de la dimensionnalité [12]. Cela nous amène à nous demander quelle est l'efficacité du RL appliqué à Nim le sera lors de la mise à l'échelle du jeu. Étant un jeu plutôt simple, Nim est facilement mis à l'échelle et adapté à l'étude de l'évolutivité.

RL peut être implémenté en utilisant différents algorithmes avec différents réglages de paramètres, ce qui laisse beaucoup de place à l'enquête. Dans la recherche précédente sur RL appliquée à Nim [11], un algorithme a été étudié. Dans cet article, nous étudions comment un phénomène légèrement différent l'algorithme fonctionnerait en comparaison.

1.1 Énoncé du problème

Nous implémenterons des agents RL en utilisant deux algorithmes afin de leur permettre de trouver le gagnant stratégie du jeu mathématique appelé Nim. Ce faisant, quel algorithme et quel ensemble de les paramètres auront-ils des performances optimales en termes de temps d'apprentissage et de précision ? Depuis la dimensionnalité est connue pour être un problème pour RL, une question importante est soulevée : dans quelle mesure Il faut plus de temps à un agent pour trouver la solution optimale à mesure que le nombre d'états augmente.

1.2 Portée

Pour réaliser ce projet, les algorithmes RL Q-learning et SARSA ont été utilisés. Il y en a d'autres algorithmes disponibles pour les problèmes RL et de nombreux domaines intéressants à étudier, cependant comme Q-learning a déjà été étudié, cela a été considéré comme une référence appropriée. SARSA est une variante du Q-learning avec une légère différence et a donc été considérée comme une solution possible et sujet de recherche intéressant.

Le principal problème étudié était l'évolutivité, étant un sujet pertinent et intéressant. Cependant, les restrictions en matière de puissance de calcul ne laissaient pas beaucoup de place à une mise à l'échelle. Comme le nombre d'états et les actions ont augmenté, les temps de calcul ont augmenté en conséquence et les simulations bientôt est devenu trop long pour être réalisé dans le cadre de ce projet. Un autre aspect du projet Le réglage des paramètres des algorithmes était soumis à des restrictions. En théorie, il existe une infinité de combinaisons de paramètres disponibles pour chaque algorithme. L'ensemble du spectre de paramètres n'ont pas été étudiées. Au lieu de cela, quelques combinaisons intéressantes ont été choisies. De plus, des mesures de performance ont été effectuées sur les données d'une simulation. Moyenne sur plusieurs exécutions n'a pas été effectuée.

2 Théorie et contexte

2.1 Apprentissage par renforcement

En RL, l'apprentissage se fait par ce qu'on appelle un agent. L'agent existe dans un environnement et est équipé de capteurs pour lui permettre d'observer et d'interagir avec l'environnement. Grâce à un processus d'essais et d'erreurs, l'agent explore son environnement, prenant des décisions sur la prochaine action à entreprendre en fonction des récompenses qu'il reçoit et de son expérience passée. À titre d'exemple, DeepMind L'agent de jeu utilise uniquement le score et les pixels de l'écran comme entrées [8]. La plupart Il est intéressant de noter que cet agent apprend différentes tâches de manière indépendante, ce qui est considéré comme un grand pas en avant vers une véritable intelligence artificielle.

RL se compose de quatre composants principaux : les fonctions de politique, de récompense et de valeur ainsi que le modèle, qui sont décrits ci-dessous et traités plus en détail par Barto [4]. La politique

guide le comportement de l'agent en décrivant quelle action entreprendre étant donné un état spécifique. Alors que apprenant à effectuer une certaine tâche, l'agent recherche une politique optimale qui fournira le agent avec la récompense maximale passant d'un état de départ à un état terminal.

La fonction de récompense attribue une récompense à l'agent soit après chaque action, soit après une série d'actions en fonction de la tâche. La fonction de récompense est là pour définir l'objectif principal du agent.

La fonction valeur est une mesure de l'avantage qu'il y a à se trouver dans un certain état ou à entreprendre une certaine action dans un certain état, si vous suivez la politique. Il contrôle si oui ou non l'agent doit rechercher un gain à court terme ou poursuivre une stratégie à long terme, et ouvre la voie à la découverte une politique optimale.

Enfin, un modèle serait constitué de toute information préalable sur l'environnement. L'agent et l'environnement peut être codé sans modèle [4, 7]. Dans le cas sans modèle, il est nécessaire que le agent pour découvrir les fonctions de valeur et de récompense en explorant l'environnement.

Dans le cas sans modèle, une question d'exploration ou d'exploitation se pose lorsqu'il s'agit de à acquérir une récompense maximale [4, 7]. L'agent peut choisir d'exploiter ce qui est déjà connu et effectuer les actions qu'il sait pour générer la récompense la plus élevée, ou il peut choisir d'explorer de nouvelles actions avec des récompenses inconnues pour avoir une chance de trouver une meilleure politique.

En combinant ces quatre composants principaux, l'agent peut apprendre lui-même la solution optimale. comportement compte tenu de tout état qu'il rencontrera.

2.2 Processus de décision markoviens

La base mathématique du RL repose sur les processus de décision de Markov (MDP) [4,7]. Un MDP se compose d'un ensemble d'états S et d'un ensemble d'actions A . Dans chaque état $s \in S$ l'agent peut exécuter l'une des nombreuses actions $a \in A$. La fonction de récompense R mappe une récompense à chaque action effectué à partir d'un État. La fonction de transition d'état T cartographie la probabilité d'arriver dans l'état s' de s après avoir agi a . De plus on fait l'hypothèse de la propriété de Markov qui stipule que le résultat d'une action dépend uniquement de l'état actuel dans lequel l'agent se trouve et est indépendant de l'histoire des actions et des états ayant conduit à l'état actuel.

Chaque état a une valeur $V(s)$ qui est la récompense attendue que l'agent recevra s'il suit la politique actuelle de cet état. Tout au long de la phase de formation, la valeur de chacun l'état $V(s)$ est mis à jour, tout comme la politique suivie par l'agent. La politique et la fonction de valeur $V(s)$ sont mis à jour simultanément. Au fil du temps, ceux-ci convergent vers V^* et π^* , les solutions optimales pour les fonctions de valeur et de politique respectivement.

2.3 Algorithmes d'apprentissage par renforcement

Les algorithmes RL recherchent des politiques optimales en mettant à jour la valeur attendue d'un état en fonction de la valeur attendue des États successeurs. En d'autres termes, ils font des estimations basées sur estimations connues sous le nom de bootstrapping [4]. Lors de la mise à jour de ces estimations de valeur, les politiques sont également mises à jour et les agents peuvent alors facilement être programmés pour suivre une politique optimale, c'est-à-dire une politique générant une récompense maximale.

Parmi les différents algorithmes RL, le Q-learning est devenu populaire [7]. Cet algorithme également comme un autre nommé SARSA, sont abordés dans cet article.

2.3.1 Q-Learning

Dans Q-Learning, la valeur attendue des états et des actions est stockée dans ce que l'on appelle la table Q. Dans la table Q, chaque élément (paire état-action) se voit attribuer une valeur $Q(s,a)$ indiquant à l'agent la récompense attendue pour avoir pris l'action a dans l'état s . Ceci est illustré dans Figure 1.

Les valeurs de la table Q sont mises à jour à l'aide de l'équation (1). Dans Q-learning, les valeurs sont mis à jour par rapport à l'action qui maximise la récompense attendue dans l'état successeur avec équation (2). Les paramètres de la formule de mise à jour (1) doivent être ajustés pour que l'agent puisse converger vers la politique optimale le plus rapidement possible.

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \max_{a'} Q(s',a') - Q(s,a)] \quad (1)$$

$$a' = \arg \max_{a'} Q(s',a') \quad (2)$$

States\Actions	1	2	3	4
1	0	-10	0	0
2	0	0	0	0
3	5	0	0	5
4	0	0	-10	0

Figure 1. Une table Q de paires état-action. Cet exemple comporte quatre états et quatre actions possibles par État. Les éléments contiennent des valeurs pour effectuer une action spécifique dans un état spécifique, par exemple la valeur -10 de l'action 2 dans l'état 1.

Le paramètre α est le paramètre de taux d'apprentissage et prend une valeur comprise entre 0 et 1. Plus α est proche est égal à 1, plus l'accent est mis sur les récompenses récentes, où $\alpha = 1$ signifie que le nouvel état- la valeur de l'action écrase complètement l'ancienne. Une valeur α de 0 signifie qu'aucun apprentissage ne nécessitera place tandis qu'une valeur proche de 0 signifie modifier légèrement les anciennes valeurs à chaque mise à jour. Ce pourrait être assimilé à la mémoire à long terme.

Le facteur d'actualisation γ prend également une valeur comprise entre 0 et 1. Si $\gamma = 0$ alors l'importance est placé sur la maximisation de la récompense actuelle et aucune propagation de la récompense ne se produira, alors que $\gamma = 1$ signifie que plus d'importance est accordée aux récompenses futures. Dans ce cas, les récompenses seront propagées des paires état-action aux paires état-action prédécesseurs [13].

L'algorithme Q-learning est plutôt simple (voir figure 1). L'agent commence dans état s , il choisit alors une action a selon une politique qui contrôle son comportement. Le Q- Le tableau est mis à jour comme décrit ci-dessus. Ce processus est ensuite répété à partir du nouvel état jusqu'à ce qu'un l'état terminal est atteint.

Pour qu'un agent apprenne de son environnement, il lui faut explorer. La politique attribué à l'agent lui indique combien d'exploration il doit faire et combien il doit exploiter les connaissances qu'il a déjà acquises, également connues sous le nom d' arbitrage exploration/exploitation [4]. Au début de la formation, il est avantageux pour l'agent d'explorer autant que possible afin de trouver quelles récompenses sont disponibles.

```

Initialize  $Q(s, a)$  arbitrarily
Repeat (for each episode):
  Initialize  $s$ 
  Repeat (for each step of episode):
    Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
    Take action  $a$ , observe  $r, s'$ 
     $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
     $s \leftarrow s'$ ;
  until  $s$  is terminal

```

Figure 2. L'algorithme Q-learning ([4]).

Une politique d'exploration standard est connue sous le nom de ϵ -gourmande [4,7]. En ϵ -gourmand, l'agent choisit la meilleure action (action avec la valeur Q la plus élevée) avec une probabilité de ϵ . À l'inverse avec probabilité $1 - \epsilon$ il choisit une action au hasard parmi une distribution uniforme. Le but de ceci La politique est d'encourager l'exploration. À mesure que le nombre de cycles d'entraînement augmente, la valeur de ϵ peut être augmenté jusqu'à ce qu'aucune autre exploration ne soit effectuée. De cette façon, l'agent explorera dans le début de son régime de formation avant de commencer à exploiter ses connaissances du milieu.

Il est intéressant de noter que le Q-learning est insensible à l'exploration. En d'autres termes, l'agent trouvera le politique optimale si elle est autorisée à visiter chaque paire état-action un nombre infini de fois [4]. Le Dans cette optique, le rôle d'une politique d'exploration est d'accélérer le processus d'apprentissage [7].

L'algorithme Q-learning est ce qu'on appelle un algorithme hors politique [7]. En d'autre En d'autres termes, l'agent estime la politique optimale tout en en suivant une autre (par exemple ϵ -gourmand).

2.3.2 SARSA

SARSA est une variante du Q-learning utilisant également une Q-table bien qu'elle diffère du Q-learning dans la mise à jour de la Q-table [7]. Alors que Q-learning met à jour la Q-table en utilisant l'état-paire d'actions avec la valeur la plus élevée, SARSA met à jour le tableau en fonction de la politique en cours suivi. De cette manière, la politique d'exploration devient une partie de la politique optimale. SARSA est connu sous le nom d' algorithme d'apprentissage sur politique [7]. L'algorithme pour SARSA est donné dans la figure 3.

```

Initialize  $Q(s, a)$  arbitrarily
Repeat (for each episode):
  Initialize  $s$ 
  Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
  Repeat (for each step of episode):
    Take action  $a$ , observe  $r, s'$ 
    Choose  $a'$  from  $s'$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
     $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$ 
     $s \leftarrow s'; a \leftarrow a';$ 
  until  $s$  is terminal

```

Figure 3. L'algorithme SARSA [4].

2.4 Nim

Le jeu de Nim est un jeu mathématique dans lequel deux joueurs se relaient pour retirer un quantité arbitraire de pions provenant d'un des tas qui composent le jeu (par exemple quatre tas de 1, 3, 5 et 7 jetons). Au moins un marqueur doit être retiré par le joueur à chaque se déplacer. À chaque mouvement, le joueur est autorisé à retirer des pions d'un seul tas. Les joueurs continuent à tour de rôle jusqu'à ce que tous les tas soient vides. Le joueur qui prend le dernier Le marqueur, c'est-à-dire qu'il laisse le plateau vide, est le gagnant (voir figure 4).

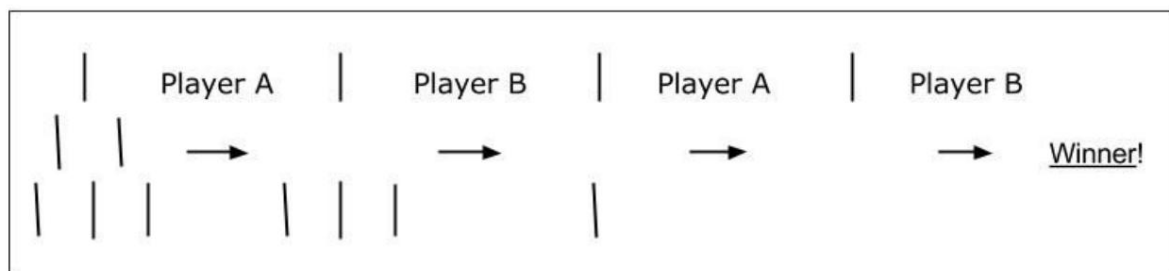


Figure 4. Un jeu complet de Nim à trois tas. Le jeu continue avec se déplace de gauche à droite. Le joueur B fait le dernier coup et gagne.

2.4.1 Somme numérique binaire et stratégie gagnante de Nim

Une somme numérique fonctionne comme une somme ordinaire à la différence que les reports sont rejetés. Ce est mieux illustré en écrivant les nombres qui doivent être additionnés les uns au-dessus des autres. En écrivant les nombres en notation binaire produisent ainsi la somme numérique binaire. Cette façon de trouver le la somme numérique est illustrée avec un jeu complet de Nim dans la figure 5.

Nim a une stratégie gagnante basée sur des sommes numériques binaires [10]. Cette stratégie gagnante est, lorsque cela est possible, effectuer des mouvements tels que la somme numérique binaire des tas devienne nulle. Une combinaison de tas avec une somme numérique binaire nulle est appelée une combinaison sûre [10]. Sortie le tableau avec une combinaison sûre de tas à chaque mouvement garantit la victoire [10]. Si joueur A effectue un mouvement en laissant une combinaison sûre, le joueur B ne peut pas le faire. Une combinaison sûre signifie qu'il y a un nombre pair de un dans chaque colonne de la somme numérique binaire, comme illustré au-dessus de. Tout mouvement valide supprime désormais un un d'au moins une colonne et rend cela impossible pour laisser une combinaison sûre. Au lieu de cela, cela laisse un nombre impair de un dans au moins un colonne qui peut facilement être compensée lors du prochain mouvement en réduisant un tas afin que chaque La colonne contient à nouveau un nombre pair de un et laisse donc à nouveau une combinaison sûre.

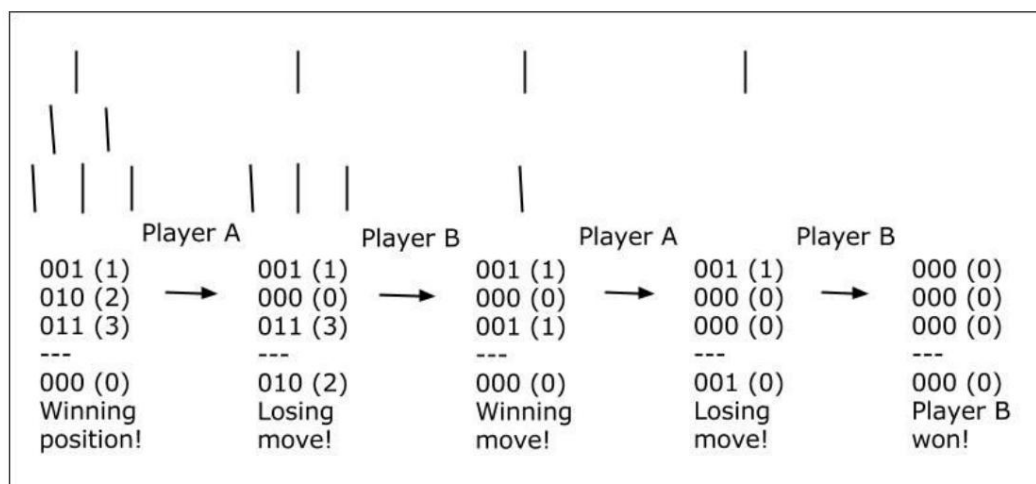


Figure 5. Un jeu complet de Nim de trois tas comprenant des tas numériques binaires

sommes. Les tas sont illustrés sous forme de rangées de compteurs et les sommes numériques binaires sont écrit sous chaque état. Le jeu se déroule par mouvements de gauche à droite.

Le joueur B fait le dernier coup et gagne.

En conséquence, le joueur commençant dans une position gagnante, ayant la possibilité de quitter le plateau avec une combinaison sûre de tas, gagnera la partie, s'il est familier avec la stratégie gagnante.

A l'inverse, si le plateau comprend une combinaison sûre au début d'un tour de jeu, le deuxième joueur gagnera s'il connaît la stratégie gagnante. L'existence de cette stratégie nous permettent d'observer les performances de l'agent lorsqu'il converge vers la solution optimale.

3. Méthodologie

Dans ce travail, l'environnement et l'agent ont été implémentés en Python 3.4.2. Pour le lecteur intéressé, le code est disponible sur <https://github.com/NimQ/NimRL.git>.

3.1 Environnement, états et actions

L'environnement du jeu de Nim est le plateau lui-même. Un État est toute permutation du conseil d'administration.

Le nombre d'états individuels possibles dans une partie de Nim peut être calculé assez facilement. Pour un tableau contenant trois tas de compteurs x_1 , x_2 et x_3 , le nombre total d'états est donné par $(x_1 + 1)(x_2 + 1)(x_3 + 1)$.

Les agents interagissent avec l'environnement en supprimant des pions du tableau. De tas de x marqueurs, un agent peut prendre n'importe quoi de 1 à x marqueurs. L'agent doit cependant ne prendre que des pions d'un seul tas à chaque mouvement. Le nombre total d'actions disponibles pour le agent est donc égal à $x_1 + x_2 + x_3$.

Sauf indication contraire, le tableau utilisé dans ce travail était composé de quatre tas avec 1, 3, 5 et 7 compteurs dans chaque tas respectivement. Le nombre total d'États dans ce cas est $(1+1)(3+1)(5+1)(7+1) = 384$. Le nombre total d'actions est donc $1+3+5+7 = 16$.

Selon Barto [4], les récompenses sont considérées comme faisant partie de l'environnement. Dans ce travail les agents recevaient une récompense de +1 s'ils gagnaient une partie, -1 s'ils perdaient et 0 pour chaque transition état pendant les jeux.

3.2 L'Agent

Au début de la formation, les entrées dans la table Q de l'agent étaient mises à zéro. Les agents étaient programmé pour lire le tableau et choisir une action selon l'un des algorithmes donnés

à la section 2.3. Si une action en justice était choisie, l'agent exécutait l'action et le conseil d'administration était modifié en conséquence. Dans Q-learning et SARSA, la politique demande parfois à l'agent pour choisir une action basée sur la paire état-action ayant la plus grande valeur. Au début de la formation lorsque de nombreuses entrées dans la table Q ont la même valeur, cela peut conduire l'agent à choisir un déplacement illégal. Dans un tel cas, le mouvement a été ignoré tandis que la table Q était mise à jour avec une valeur de -10 et la planche est restée inchangée, un peu comme si on se cognait les doigts en atteignant pour un biscuit. L'agent a alors choisi un nouveau coup valide. De cette façon, l'agent a ignoré ces actions à l'avenir et comme ces actions invalides ne pourront jamais être choisies, elles n'ont aucune incidence sur la capacité d'apprentissage de l'agent.

La récompense était remise à l'agent après chaque paire de mouvements : l'agent bougeait, puis le l'adversaire s'est déplacé, de cette façon l'agent a vu les conséquences ultimes de ses actions et de son Q-tableau a été mis à jour. Si par contre l'adversaire a fait le premier coup de la partie, le La table Q de l'agent a été mise à jour pour la première fois après que l'adversaire ait effectué son deuxième mouvement.

Dans ce travail, il existe deux critères que l'on peut observer lors de la recherche d'un comportement optimal dans l'agent : soit le pourcentage de matchs gagnés, soit la fréquence à laquelle l'agent effectue un coffre-fort mouvement combiné quand cela est possible. Une combinaison sûre est vérifiée dans le code avec le Opérateur XOR. L'opérateur XOR renvoie 0 si les tas totalisent une combinaison sûre et 1 si pas.

3.3 Formation de l'agent

Plusieurs méthodes de formation des agents ont été examinées. Ceux-ci sont répertoriés dans les sections ci-dessous.

Dans Nim, la configuration initiale du plateau peut laisser le joueur qui commence dans une position avantageuse ou désavantageuse [10]. Pour éviter que cela devienne un facteur limitant Dans le processus d'apprentissage de l'agent, le joueur qui ferait le premier pas était choisi au hasard. De plus, le générateur aléatoire a été optimisé pour assurer une meilleure comparaison des résultats. Dans Pour découvrir quel ensemble de paramètres était le meilleur pour la tâche à accomplir, les agents ont été testés sur un grand nombre de jeux d'entraînement pour différents réglages. Ces réglages étaient tous possibles combinaisons de $\alpha = \{0, 1, 0,5, 0,99\}$, $\gamma = \{0, 1, 0,5, 1\}$ et $\epsilon = \{0,2, 0,8\}$, ϵ étant soit constante, ϵ_c , ou augmentant régulièrement au cours du temps ϵ_i .

3.3.1 Agent contre intelligent

L'adversaire intelligent a joué un jeu « parfait ». Si un coup était possible qui laissait le plateau avec une somme binaire de 0, en d'autres termes un coup gagnant, alors l'adversaire malin ferait cela se déplacer. Si un coup gagnant n'était pas possible, l'adversaire intelligent effectuait un coup aléatoire.

3.3.2 Agent vs aléatoire

De manière analogue à la méthode ci-dessus, les agents étaient également entraînés contre un adversaire qui faisait seulement des mouvements aléatoires. C'est-à-dire que l'adversaire a pris un nombre valide de pions au hasard. tas non vide choisi sur le tableau.

3.3.3 Agent contre Agent

Dans ce régime, les algorithmes Q-learning et SARSA ont subi un entraînement les uns contre les autres.

3.4 Évaluation de l'agent

À intervalles réguliers, les agents étaient testés en rivalisant avec l'adversaire intelligent pendant 250 jeux en utilisant leurs dernières estimations de la politique optimale. De ces jeux, l'agent le taux de gain et le ratio de mouvements laissant une combinaison sûre lorsque cela était possible ont été enregistrés. Comme mentionné à la section 2.4.1, un agent peut démarrer dans une position gagnante. À partir de cette victoire La position est la seule chance qu'a l'agent de gagner contre l'adversaire intelligent. Pour cette raison, le taux de victoire a été défini comme le nombre de parties gagnées par rapport au nombre de parties commençant dans une position si gagnante. De plus, à partir de la section 2.4.1, il est connu que faire des mouvements optimaux garantir de rester en position gagnante et éventuellement de gagner la partie. Ainsi le rapport de les coups gagnants ont été définis comme le nombre de coups gagnants effectués par rapport au nombre de fois qu'un Le coup gagnant était une action possible.

Les tracés du ratio de coups gagnants en fonction du nombre de matchs d'entraînement ont été étudiés comme un mesure de performance puisqu'une valeur de 1 pour ce ratio garantit que les mouvements optimaux sont toujours prises et donc qu'une stratégie gagnante a été apprise.

3.5 Mise à l'échelle

Avec l'environnement et l'agent déjà implémentés, la mise à l'échelle du jeu était simple. Avec un Un nombre croissant de tas, les régimes de formation mentionnés ci-dessus ont été répétés. Plus de tas ont été ajoutés au tableau par étapes en le faisant passer de $\{1,3,5,7\}$ à $\{1,3,5,7,9,11\}$. En ajoutant plus ajoute au jeu le nombre total de matchs d'entraînement et l'intervalle régulier auquel les agents ont été testés a été allongé.

4 Résultats et discussion

4.1 Sélection des paramètres

Les paramètres α , γ et ϵ jouent un rôle très important dans la rapidité avec laquelle l'agent trouve le politique optimale. Cependant, le réglage des paramètres est un processus très long et inexact. La difficulté réside dans la recherche d'une tendance dans la façon dont différentes combinaisons de paramètres affecter le processus d'apprentissage.

Avant la formation, une présomption était faite. L'environnement Nim est statique dans le sens qu'un mouvement optimal sera toujours un mouvement optimal. Une paire état-action représentant un Le mouvement optimal dans la table Q doit donc conserver une valeur élevée une fois acquis. Cela suggère de fixer α plutôt bas (par exemple $\alpha = 0,1$).

Considérant à quoi ressemble l'environnement Nim, le but est de gagner et c'est là que le l'agent est récompensé - il serait raisonnable de mettre l'accent sur les récompenses futures pour s'améliorer rapidement performances du jeu. Cela signifie qu'un $\gamma = 1$ devrait donner les meilleurs résultats.

Une faible valeur de ϵ serait considérée comme une bonne chose en début d'entraînement pour faire assurez-vous que tous les états (ou au moins autant que possible) sont visités par l'agent d'apprentissage. Cependant, plus tard pendant la formation, l'agent doit avoir rencontré suffisamment de situations pour comprendre les

une stratégie et une exploration optimales conduiraient désormais à faire de mauvais mouvements lorsque la stratégie est déjà connu. Pour éviter cela, ϵ doit être augmenté au fil du temps jusqu'à 1.

En analysant les données collectées, les tendances au sein de la sélection des paramètres ont été discernées. Un exemple de la façon dont cela est apparu est montré dans la figure 6. La meilleure combinaison de paramètres provoque l'agent apprendra le plus rapidement et convergera ainsi vers un ratio de mouvements gagnants de 1 le plus rapidement. Figure 7 compare l'effet sur la convergence d'un bon ensemble de paramètres avec un moins bon ensemble de paramètres paramètres.

Nous avons constaté que nos présomptions étaient quelque peu vérifiées. Autrement dit, α devrait être faible et γ élevé. Le paramètre contrôlant l'exploration ϵ s'est avéré fournir les meilleurs résultats lorsqu'il est plutôt ϵ élevé et croissant plutôt que ϵ constant.

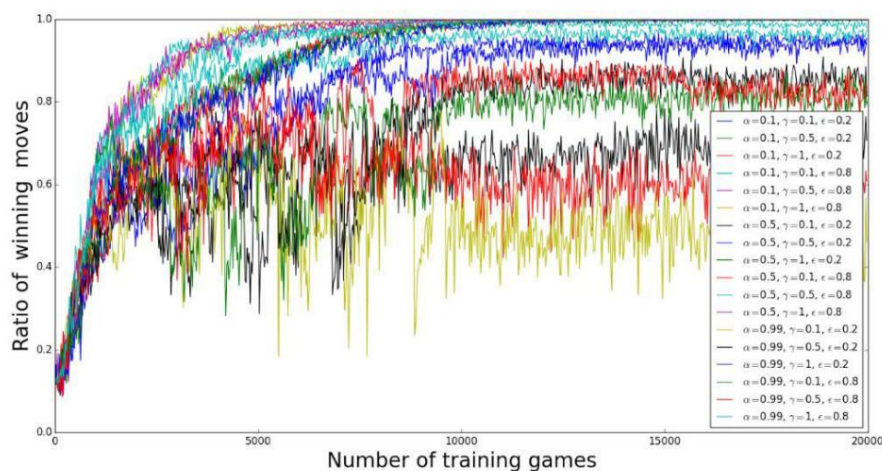


Figure 6. Graphique des performances de l'agent SARSA avec tous les ensembles de paramètres entraînés contre un adversaire intelligent sur un plateau de quatre tas. Certains groupes similaires Les ensembles de paramètres ont montré de meilleures performances que les autres.

Une autre difficulté a été constatée lors de l'activation ou de la désactivation de la fonction d'amorçage du générateur aléatoire. L'idée était que les agents avec différentes combinaisons de paramètres seraient confrontés au même modèle de nombres créés par le générateur aléatoire. A partir de là, on peut comparer les performances de différentes combinaisons de paramètres dans à peu près les mêmes situations. Cependant, une fois le hasard Le générateur n'a pas été amorcé, des différences notables de convergence ont pu être observées pour le même combinaisons de paramètres comme illustré dans la figure 8. Ce comportement a été observé à la fois dans le Q-apprentissage et algorithmes SARSA. Cela peut être considéré comme une preuve de la difficulté de réglage paramètres pour ces deux algorithmes et suggère que les différences entre les Les graphiques pour différents paramètres pourraient être dus à des différences dans le déroulement de la formation plutôt qu'à

en raison uniquement des valeurs de paramètres modifiées. Une solution à ce problème pourrait être de prendre une moyenne sur plusieurs simulations de la même combinaison de paramètres. Cependant, cela augmenterait considérablement les temps de simulation et n'a pas été réalisée dans ce travail.

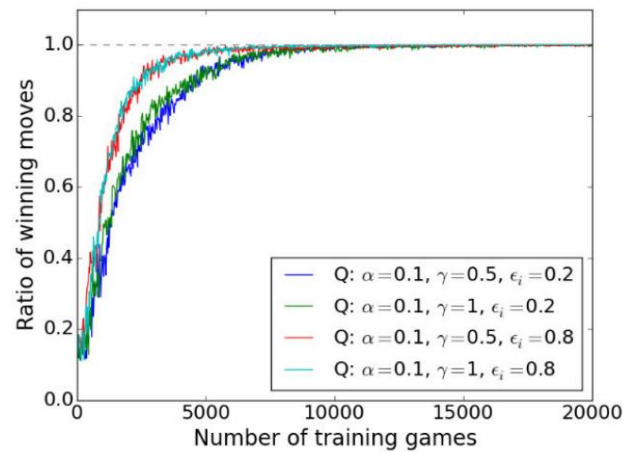


Figure 7. Graphiques des agents Q-learning entraînés contre l'adversaire intelligent sur un tableau de quatre tas utilisant des combinaisons de paramètres bonnes et moins bonnes.

Comme l'ensemble optimal de paramètres est inconnu et qu'il est difficile de trouver celui qui se demande si possible, cela peut être une tâche à accomplir par un agent RL. Serait-il possible d'avoir des agents RL apprendre à trouver les paramètres optimaux ?

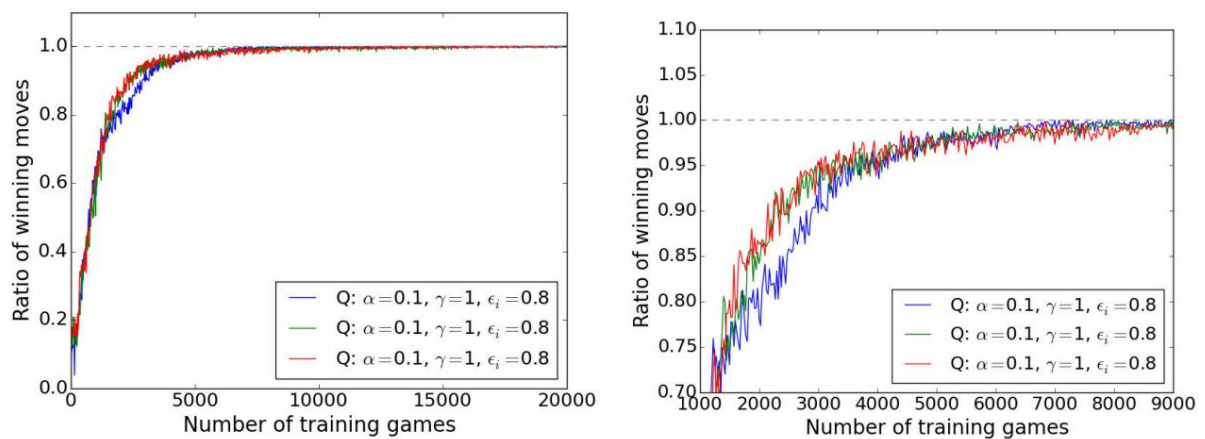


Figure 8. Graphiques d'un agent Q-learning entraîné contre un adversaire intelligent sur un tableau de quatre tas utilisant le même ensemble de paramètres pour trois simulations de formation. Le tracé de droite est un gros plan du tracé de gauche. On peut voir du hasard dans performances entre les différentes simulations.

4.2 Agent contre intelligent

Suite à la méthode de formation décrite dans la section 3.3, les meilleurs paramètres se sont avérés être proche de ce qui était présumé dans la section 4.1. Pour Q-learning et SARSA, ces valeurs étaient $\alpha = 0,1$, $\gamma = 1$ et $\epsilon = 0,8$, ϵ augmentant jusqu'à 1 après 10 000 parties. Une comparaison du Q-learning et les performances des algorithmes SARSA sont illustrées à la figure 9.

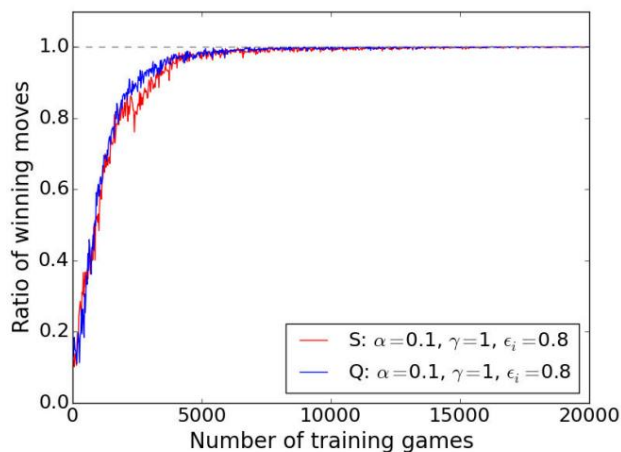


Figure 9. Graphiques du ratio de coups gagnants des agents Q et SARSA par rapport au nombre de jeux d'entraînement lorsque vous vous entraînez contre un adversaire intelligent sur un plateau de quatre des tas.

De la figure 9, on peut voir que la convergence vers 1 se produit autour de 10 000 jeux d'entraînement pour les deux algorithmes, bien qu'il semble que SARSA soit quelque peu en retard par rapport au Q-learning pour atteindre la stratégie optimale. Il convient toutefois de noter qu'il ne s'agit pas là des valeurs optimales définitives. paramètres de ces algorithmes, ce sont plutôt les paramètres optimaux de toutes les combinaisons mis en œuvre dans ce projet.

4.3 Agent contre aléatoire

Lorsque l'agent RL s'entraînait contre un adversaire effectuant des mouvements aléatoires, il ont constaté que les performances étaient inférieures à celles d'un agent s'entraînant contre un adversaire intelligent (voir figure 10). La vitesse de convergence (si jamais elle avait appris complètement) était évidemment bien inférieure. Ce peut s'expliquer par la nature de l'environnement pour cette tâche. Puisque l'agent s'est entraîné contre un adversaire effectuant des mouvements aléatoires, l'agent gagnerait de nombreuses parties en effectuant des mouvements non aléatoires.

mouvements optimaux. En effectuant ces mouvements non optimaux et en gagnant quand même, l'agent se propagerait récompense positive aux mouvements non optimaux, et ainsi rater la stratégie optimale. L'agent serait confus quant à la raison pour laquelle il a été récompensé. Le plus remarquable est que l'agent Q-learning a presque atteint la convergence, mais avec une valeur de ϵ inférieure à celle du régime agent contre intelligent, ce qui signifie une exploration beaucoup plus approfondie de l'espace étatique était nécessaire.

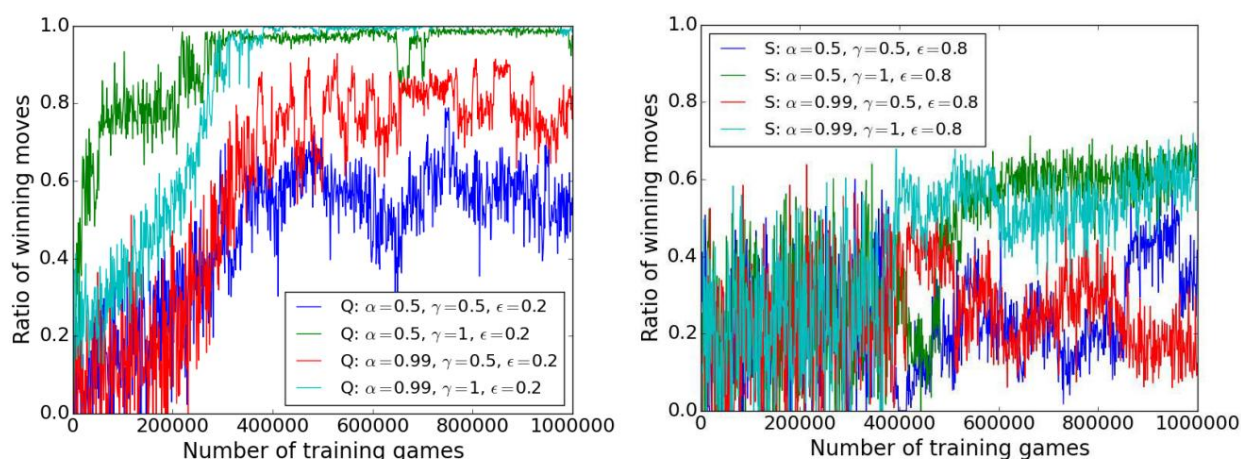


Figure 10. Performances des agents Q-learning (à gauche) et SARSA (à droite) avec différents ensembles de paramètres (ϵ atteint 1 à 400 000 jeux d'entraînement) entraînés contre un adversaire effectuant des mouvements aléatoires. Convergence vers un comportement optimal pourrait être vu pour certains des ensembles de paramètres de l'agent Q-learning. SARSA n'a pas convergé vers un comportement optimal.

Malheureusement, ce régime de formation n'a pas fourni des niveaux de performance satisfaisants sur un plateau de quatre tas. Par conséquent, ce régime de formation a été abandonné lors de l'intensification le nombre de tas.

4.4 Agent contre Agent

Une comparaison du comportement d'un agent SARSA et d'un agent Q-learning formé en jouant l'un contre l'autre et testés contre l'adversaire intelligent est illustré à la figure 11. Les deux agents atteignent la convergence après environ 8 000 matchs d'entraînement. De là, on ne peut pas observer une grande différence dans les performances des algorithmes.

Un comportement très intéressant s'est produit dans ce régime de formation. Contrairement à la figure 9 de Section 4.2 Agent contre Smart, le ratio de coups gagnants des agents ne fluctue plus en dessous de 1. Lorsque atteignant un comportement optimal, les agents ont continué à faire des mouvements optimaux lorsque cela était possible pour le reste de la simulation. Les agents ont mieux appris dans ce régime de jeu contre un adversaire qui était aussi en train d'apprendre le jeu que dans les deux régimes précédents.

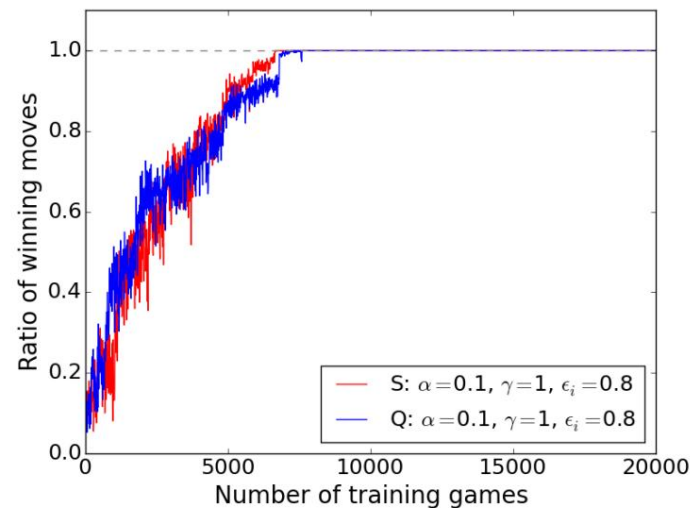


Figure 11. Performances de Q et SARSA lorsqu'ils sont entraînés l'un contre l'autre dans un jeu de quatre tas utilisant le même ensemble de paramètres (ϵ atteint 1 à 10000 entraînements Jeux). Les deux agents apprennent le comportement optimal à peu près en même temps. Une fois le comportement optimal est appris, les deux agents continuent de jouer de manière optimale.

4.5 Mise à l'échelle

La mise à l'échelle de Nim vers des conseils d'administration plus grands a augmenté le temps d'apprentissage d'un agent. Ceci est illustré pour les planches de cinq et six tas respectivement dans la figure 12, montrant les tracés des performances du Q-apprentissage et agents SARSA entraînés contre l'adversaire intelligent. Dans ce régime, au sein d'un conseil d'administration cinq tas, l'agent Q-learning a convergé vers un ratio de coups gagnants de 1 à environ 13 000 jeux d'entraînement. SARSA était un peu plus lent, tout comme dans le cas de quatre tas, et a convergé vers 1 après environ 17 000 matchs d'entraînement. Sur un plateau de six tas, SARSA a grimpé vers la stratégie optimale avec une pente plus douce, tout en s'installant avec un ratio de coups gagnants de 1 avant l'agent Q-learning qui a convergé plus rapidement mais a continué à fluctuer légèrement. Pour les deux agents, le comportement optimal s'est produit à environ 1,5 106 matchs d'entraînement. Pour une meilleure perspective sur l'augmentation du temps d'apprentissage avec des tas ajoutés, la convergence pour quatre, cinq et six tas respectivement sont représentés sur la figure 13.

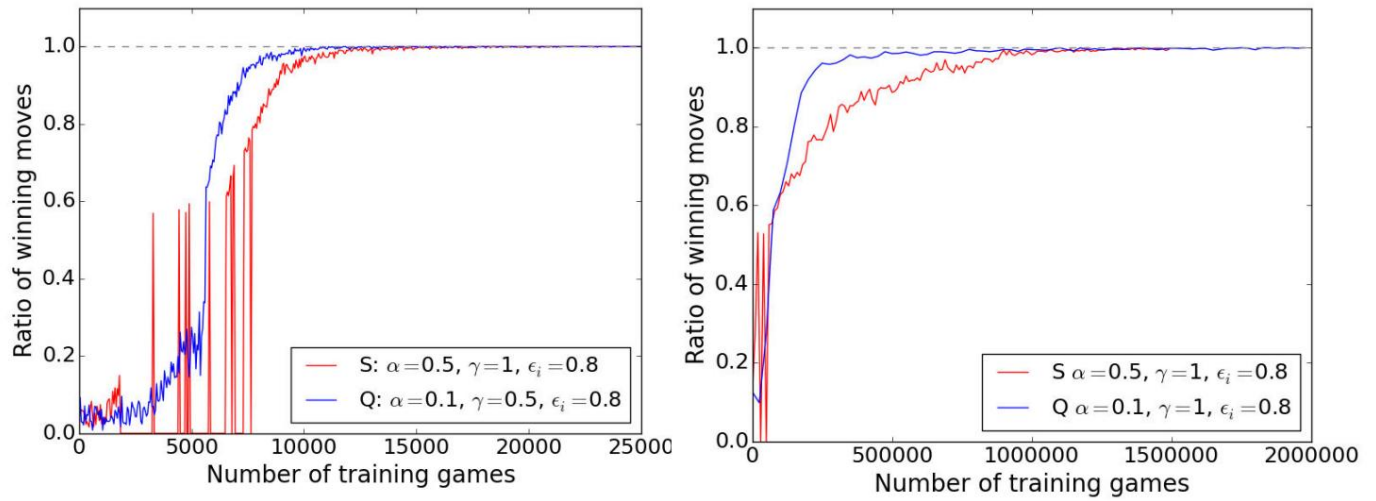


Figure 12. Agents Q et SARSA vs smart sur des cartes de 5 tas (à gauche)
et 6 tas (à droite).

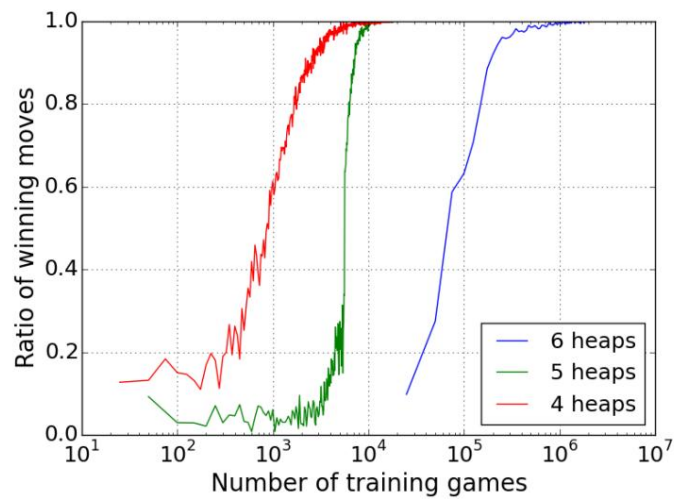


Figure 13. Graphiques de Q-learning par rapport à Smart pour des tableaux de quatre, cinq et six tas montrant le ratio de coups gagnants par rapport au nombre de jeux d'entraînement sur un échelle logarithmique. Le temps de convergence est beaucoup plus élevé sur un tableau de six tas.

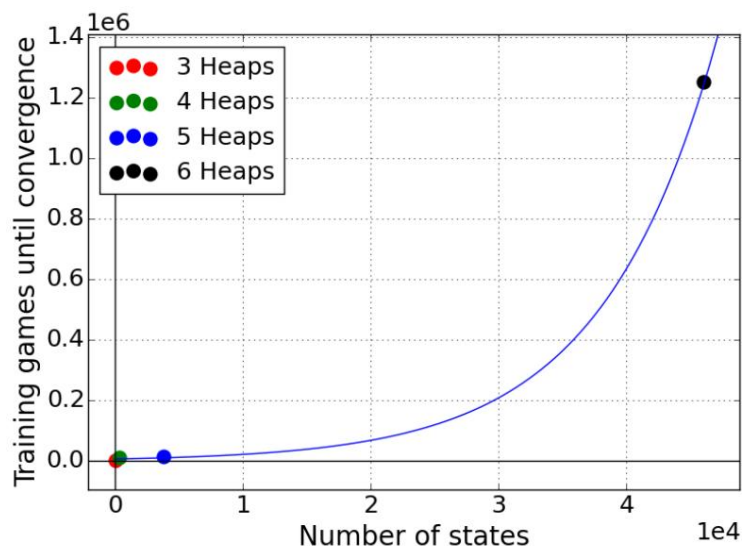


Figure 14. Graphique montrant la relation exponentielle entre le nombre d'états et nombre de parties nécessaires pour qu'un agent converge vers un ratio de coups gagnants de 1.

Afin d'étudier l'évolution du temps de formation en fonction du nombre d'états, les données ont été ajoutées à partir d'une simulation de Q vs. Smart sur un tableau de trois tas de un, trois et cinq compteurs. La convergence sur ce plateau à trois tas n'était que de l'ordre de plusieurs centaines de jeux d'entraînement. Les données du nombre d'états par conseil d'administration tracées au nombre de formations les jeux requis pour atteindre des performances optimales correspondent à une courbe exponentielle (voir figure 14). Cela suggère que les agents deviennent rapidement plutôt inefficaces pour trouver la stratégie optimale lorsque le jeu est étendu.

Formation d'agents Q-learning contre des agents SARSA sur des tableaux de plusieurs tas montrés des résultats similaires à ceux de Q vs. Smart. Le temps de convergence a augmenté avec l'ajout de tas et est vite devenu inefficace. Il est intéressant de noter que dans certains cas, pour un tableau de cinq tas, on pourrait voir le temps de convergence d'un agent très proche de celui d'un agent apprenant sur un tableau de quatre tas. Dans ces cas, les performances de l'autre agent étaient inférieures, voir le tracé des cinq tas dans la figure 15. Cela suggère que les agents pourraient éventuellement être couplés de manière à ce que l'un apprenne plus rapidement tandis que l'autre fonctionne comme un catalyseur. Cependant, ces résultats étaient clairement soumis à la façon dont les séances de formation s'est avérées. Comme mentionné à la section 4.1, la séquence de nombres aléatoires peut affecter la performance de l'agent au cours de sa période de formation. Relancer l'entraînement avec un autre

la valeur de la graine dans le générateur de nombres aléatoires a parfois entraîné une surperformance de SARSA

Q-apprentissage. Il est donc difficile de tirer des conclusions sur les effets de la mise à l'échelle dans ce régime.

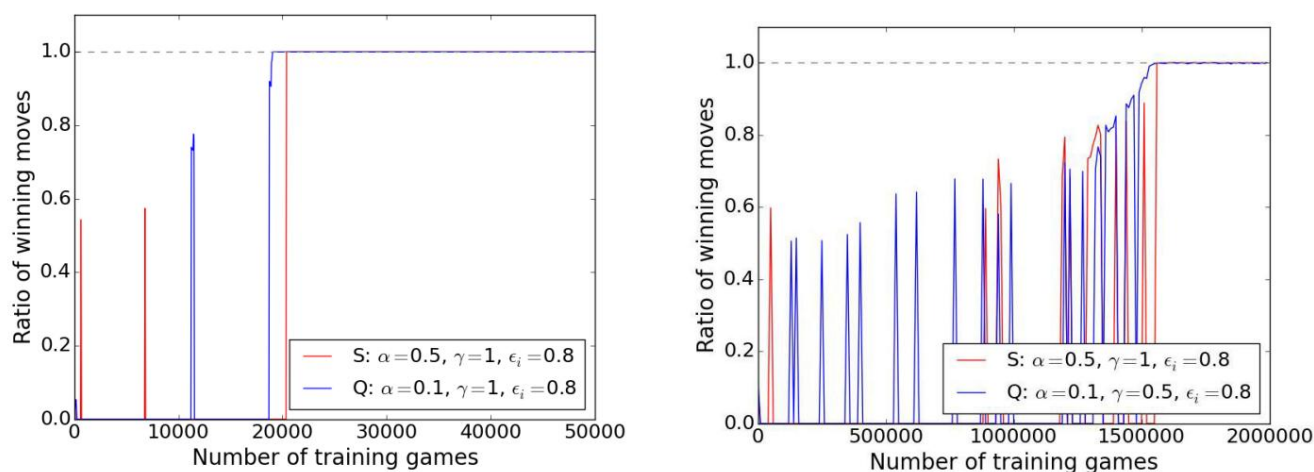


Figure 15. Q et SARSA s'entraînent l'un contre l'autre sur un plateau de cinq tas

(à gauche) et six tas (à droite). Les paramètres pour les deux agents sont les meilleurs obtenus à partir de s'entraîner contre un adversaire intelligent. Les deux agents convergent vers un comportement optimal. Sur sur le tableau à cinq tas (tracé de gauche), l'agent Q converge rapidement tandis que l'agent SARSA converge plus lentement.

Comme dans le cas des tableaux de quatre tas, une fois la stratégie optimale trouvée, les agents a continué à jouer avec un taux de déplacement optimal de 100 %. Cependant, un comportement particulier pourrait être noté pour les deux algorithmes lors de l'entraînement les uns contre les autres sur des plateaux de 5 et 6 tas. A l'heure la performance d'un agent a chuté considérablement jusqu'à un ratio de 0 mouvement optimal avant d'augmenter soudainement vers la convergence. Le comportement exponentiel observé dans la figure 14 peut s'expliquer par examiner le nombre d'entrées dans la table Q à mesure que le nombre de tas sur le tableau augmente (voir tableau 1). Nim est un jeu simple avec une stratégie qu'un agent RL peut apprendre, mais apparemment l'action triviale d'ajouter des tas supplémentaires au tableau pose rapidement des problèmes en termes de temps de calcul requis et stockage de grandes matrices dans la mémoire d'un ordinateur.

Un remède possible pour réduire le temps de convergence lors de l'ajout de tas supplémentaires pourrait être d'utiliser les valeurs d'état-action d'un agent ayant déjà suivi une formation sur des sujets plus petits. conseils d'administration et connaît la stratégie optimale. Les mouvements optimaux seront toujours des mouvements optimaux quel que soit le nombre de tas sur le plateau. En d'autres termes, former l'agent par étapes peut accélérer le processus d'apprentissage.

Tableau 1. Augmentation de la taille de la table Q correspondant à la taille du tableau.

Conseil	États	Actions	Entrées dans la table Q
1,3,5,7	384	16	6144
1,3,5,7,9	3840	25	96 000
1,3,5,7,9,11	46 080	275	$1,27 \cdot 10^7$
1,3,5,7,9,11,13	645 120	3575	$2,31 \cdot 10^9$
1,3,5,7,9,11,13,15	10 321 920	53 625	$5,5 \cdot 10^{11}$

4.6 Autres observations et remarques

Un problème connu dans RL est que, lors d'un épisode d'entraînement, il faut attribuer un crédit au agent [7]. Au cours de la phase initiale de programmation, la table Q a été mise à jour une fois que l'agent a a terminé son coup mais avant celui de son adversaire. Nous avons toutefois constaté que l'agent n'était pas apprenant de manière optimale et gagnerait au maximum environ la moitié de ses matchs, même contre un naïf joueur aléatoire. Ce résultat nous a amené à laisser plutôt l'agent voir les conséquences de ses actions et mettez à jour la table Q après le mouvement de l'adversaire. Cette légère modification a permis à l'agent de apprendre la stratégie optimale.

Il est clair que les agents RL peuvent apprendre la stratégie optimale de Nim, même si les performances diffère selon les régimes de formation. Face à un adversaire utilisant la stratégie optimale les agents ont vite appris quelles actions sont optimales. Cependant, on peut argumenter sur la façon dont représentatif d'un problème RL pertinent, ce régime est puisque la stratégie optimale est déjà connu et mis en œuvre, alors que RL a été classé comme adapté à la résolution de problèmes avec solutions inconnues. Pourtant, il a été démontré que les agents RL s'appliquaient à des problèmes avec des les solutions peuvent découvrir des solutions complètement nouvelles [6]. De plus, RL peut être pertinent lorsque une solution est connue, mais difficile à mettre en œuvre comme dans le cas du robot retourneur de crêpes [5]. Ces exemples suggèrent que le régime Agent vs. Smart pourrait être d'une certaine pertinence.

Pour les problèmes dont les solutions sont totalement inconnues, les deux autres régimes de formation sont plus adaptés. pertinent puisque dans ces régimes, les deux joueurs commencent sans savoir comment la partie est gagnée.

Malheureusement, les résultats du régime agent vs aléatoire n'ont pas fourni des niveaux satisfaisants.

de performances. Cependant, en laissant Q et SARSA s'entraîner l'un contre l'autre, cela

Il a été constaté que les performances étaient encore meilleures que lorsque les agents étaient formés aux systèmes intelligents.

adversaire. Ce résultat est très réjouissant puisqu'il montre que deux agents RL peuvent être joués contre

les uns les autres pour trouver la stratégie optimale, même si elle était inconnue auparavant.

Dans les cas de formation et d'évaluation des performances des agents dans le cadre Agent vs. Agent

régime avec plus de quatre tas, il semble que l'agent souffre de quelque chose qui s'apparente à

« amnésie » (voir figures 12 et 13). Cela peut être dû au vaste espace étatique et au grand nombre d'États.

paires d'actions ayant la même valeur dans la table Q tout en apprenant contre un

l'adversaire effectue parfois des mouvements sous-optimaux. Il est possible qu'après avoir appris un

une stratégie assez bonne qu'un agent continue d'explorer alors qu'il aurait dû faire une stratégie optimale

se déplacer. De cette manière, il écrase et réduit la valeur de ce qui était auparavant de bonnes actions.

prendre dans un état donné.

Comme mentionné dans 1 Introduction, RL est basé sur le behaviorisme [1]. Avec les résultats en

Attention, on peut se demander s'il s'agit vraiment d'un bon modèle d'apprentissage animal. C'est vrai que ça

fonctionne pour un programme informatique. Or, le cerveau humain est un organisme très complexe. R.L.

peut être considéré comme applicable comme modèle pour des cas simples : par exemple enseigner à un très jeune

enfant comment se comporter ou apprendre à faire du vélo. En termes de tâches plus complexes telles que l'apprentissage

un instrument de musique, il est plus difficile de voir d'où proviennent les récompenses immédiates ou même les récompenses à long terme

s'adapter à un tel scénario. Les agents de RL ont réclamé un très grand nombre de jeux d'entraînement pour apprendre

une stratégie optimale lors de la mise à l'échelle. Cependant, lors de l'apprentissage d'une tâche ou d'une compétence compliquée,

les humains nécessitent également une grande quantité de pratique.

5. Conclusion

Il est tout à fait possible pour un agent RL d'apprendre la stratégie optimale de Nim dans les trois formations.

régimes traités dans cet article. Ceci vérifie les travaux précédents [11] et les développe avec l'utilisation

de SARSA en tant qu'algorithme RL appliqué à un agent apprenant Nim.

Le réglage des paramètres était difficile, mais l'hypothèse initiale d'un α faible et d'un γ élevé était

confirmé alors qu'il a été constaté que ϵ devait prendre une valeur plutôt élevée.

S'entraîner contre un adversaire intelligent était initialement considéré comme le meilleur moyen pour les agents

pour apprendre la stratégie gagnante. Cependant, ce régime de formation n'est pas très adapté aux problèmes

sans solution connue auparavant, bien que cela puisse être pertinent pour des problèmes pour lesquels un

solution est connue mais trop compliquée à mettre en œuvre ou pour trouver des solutions alternatives à problèmes. Néanmoins, il s'est avéré que lors de la formation des agents Q-learning contre SARSA

L'apprentissage des agents était non seulement plus rapide, mais les agents fonctionnaient également avec une plus grande stabilité une fois la stratégie optimale a été apprise. Ce régime de formation est plus représentatif des problèmes sans solution connue auparavant. Cependant, ce résultat était extrêmement sensible à la façon dont les jeux étaient joué pendant l'entraînement et les résultats n'étaient clairs que sur un plateau de 4 tas.

La malédiction de la dimensionnalité s'avère être un terme approprié pour décrire ce qui arrive à Nim. à mesure que le nombre de tas augmente. La croissance exponentielle de l'espace étatique rend temps d'apprentissage pour un agent incroyablement long. Pour cette raison, il peut être intéressant d'effectuer recherche sur la formation d'un agent par étapes, à l'aide de tables Q à partir de problèmes dimensionnels plus petits et en s'appuyant sur ces connaissances antérieures lors de la mise à l'échelle.

6 références

- [1] Beierholm, UR (2014). Optimalité bayésienne de la perception, de l'action et de l'apprentissage humains : preuves comportementales et neuronales. Dans *Informatique inspirée du cerveau* (pp. 117-129). Éditions internationales Springer.
- [2] Beck, JM, Ma, WJ, Kiani, R., Hanks, T., Churchland, AK, Roitman, J., ... et Pouget, A. (2008). Codes de population probabilistes pour la prise de décision bayésienne. *Neurone*, 60(6), 1142-1152.
- [3] Berkes, P., Orbán, G., Lengyel, M. et Fiser, J. (2011). L'activité corticale spontanée révèle les caractéristiques d'un modèle interne optimal de l'environnement. *Science*, 331 (6013), 83-87.
- [4] Barto, AG (1998). Apprentissage par renforcement : une introduction. Presse du MIT.
- [5] Kormushev, P., Calinon, S. et Caldwell, DG (2013). Apprentissage par renforcement en robotique : applications et défis du monde réel. *Robotique*, 2(3), 122-148.
- [6] Tesauro, G. (1995). Apprentissage des différences temporelles et TD-Gammon. *Communications de l'ACM*, 38(3), 58-68.
- [7] van Otterlo, M. et Wiering, M. (2012). Apprentissage par renforcement et processus de décision markoviens. Dans *Apprentissage par renforcement* (pp. 3-42). Springer Berlin-Heidelberg.
- [8] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, AA, Veness, J., Bellemare, MG, ... et Hassabis, D. (2015). Contrôle au niveau humain grâce à un apprentissage par renforcement profond. *Nature*, 518(7540), 529-533.
- [9] Grundy, PM (1938). Mathématiques et jeux. *Eurêka*, 2(6-8)
- [10] Bouton, CL (1901). Nim, un jeu avec une théorie mathématique complète. *Les Annales des Mathématiques*, 3(1/4), 35-39.
- [11] Järleberg, E. (2011). Apprentissage par renforcement sur le jeu combinatoire de Nim.
- [12] Gosavi, A. (2009). Apprentissage par renforcement : une enquête tutorielle et des avancées récentes. *INFORME Journal sur l'informatique*, 21(2), 178-192.

[13] Nissen, S. (2007). Apprentissage par renforcement à grande échelle utilisant q-sarsa (λ) et réseaux de neurones en cascade. Thèse de maîtrise non publiée, Département d'informatique, Université de Copenhague, København, Danemark.